CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

ID : 24IT004

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical 12: CI/CD Pipeline with GitHub Actions

Student Name: Krish Bhingradiya

Student ID: 24IT004

GitHub Repository: https://github.com/krishbhingradiya/API-Testing.git

Tech Stack: Node.js & Express

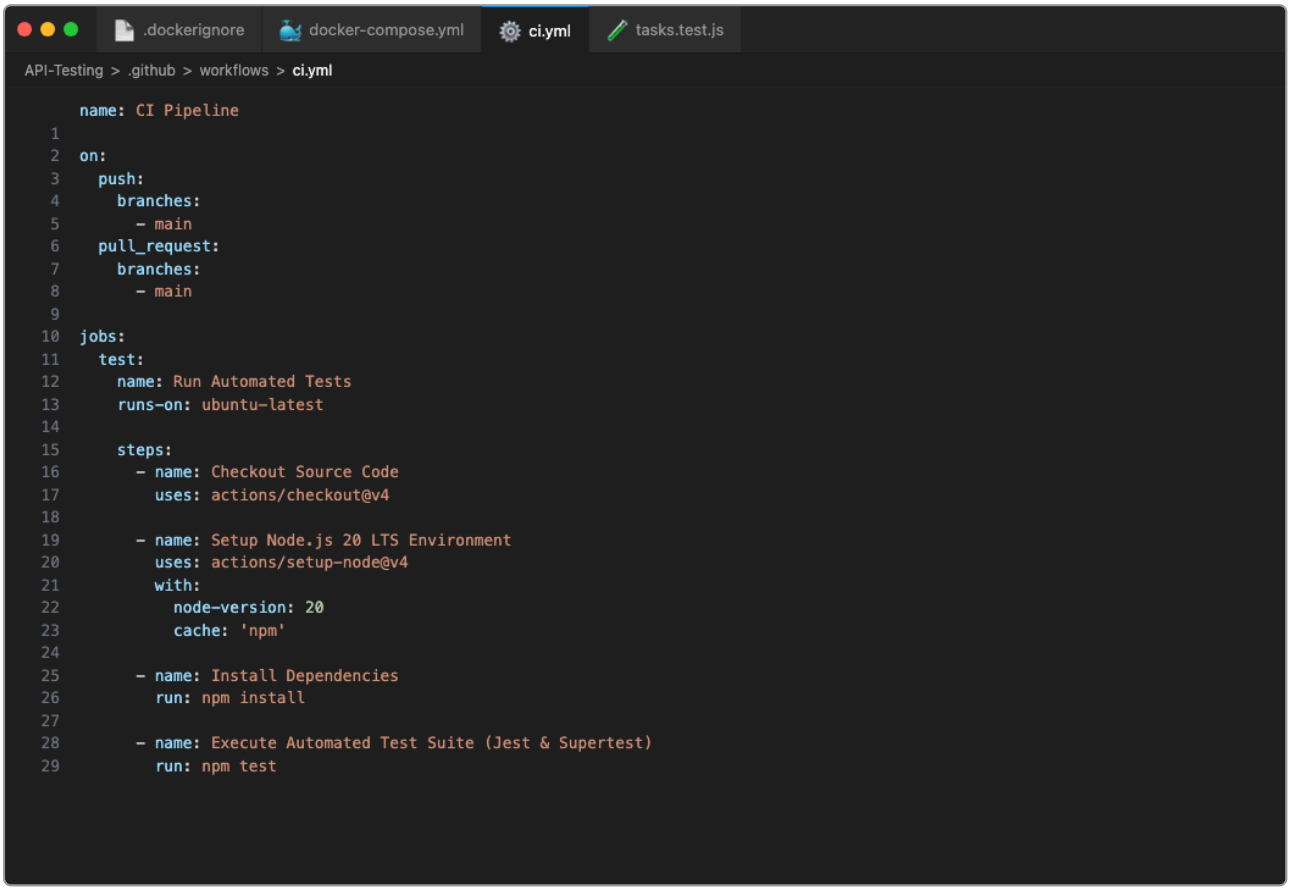
Problem Statement :

- Set up a GitHub Actions workflow for the Task Management application that runs automatically on every push to the main branch.
- The pipeline must install dependencies (npm install) as a defined step.
- The pipeline must run at least one automated test and report a clear pass/fail status.
- Demonstrate a failed pipeline caused by a deliberately broken test.
- Demonstrate a fixed pipeline that passes after the test is corrected.

Screenshots & Terminal Execution

Step – 1 : GitHub Actions Workflow Code

The GitHub Actions workflow file is configured in `.github/workflows/ci.yml`. It listens for push events on the `main` branch, checks out source code, sets up Node.js 20 LTS, installs packages, and executes the Jest test suite.



Step – 2 : Automated Test code

Automated API tests are implemented in `tests/tasks.test.js` using Jest and Supertest. In-memory caching and Mongoose query mocks are applied to test real Express endpoints without external database latency.

ci.yml | tasks.test.js | server.js

API-Testing > tests > tasks.test.js

```
const request = require('supertest');
const app = require('../server');
const Task = require('../src/models/Task');
const cache = require('../src/utils/cache');

describe('Task Manager REST API & CI Automated Test Suite', () => {
  beforeEach(() => {
    // Reset cache and stats before each test to maintain strict test isolation
    cache.flushAll();
    cache.resetCustomStats();
    jest.restoreAllMocks();
  });

  afterAll(() => {
    cache.flushAll();
    jest.restoreAllMocks();
  });

  describe('GET /tasks', () => {
    it('should return HTTP 200 with list of tasks and HATEOAS collection links', async () => {
      const mockTask = {
        _id: '507f1f77bcf86cd799439011',
        title: 'Complete Practical 12',
        description: 'Set up GitHub Actions CI/CD Pipeline',
        status: 'pending',
        priority: 'high',
        assignedUser: 'DevOps Engineer',
        createdAt: new Date().toISOString(),
        updatedAt: new Date().toISOString()
      };

      jest.spyOn(Task, 'countDocuments').mockResolvedValue(1);
      const queryMock = {
```

tasks.test.js

API-Testing > tests > tasks.test.js (Endpoints & Assertions)

```
        sort: jest.fn().mockReturnThis(),
        skip: jest.fn().mockReturnThis(),
        limit: jest.fn().mockResolvedValue([mockTask])
      };
      jest.spyOn(Task, 'find').mockReturnValue(queryMock);

      const res = await request(app)
        .get('/tasks')
        .expect('Content-Type', /json/)
        .expect(200);

      expect(res.body.success).toBe(true);
      expect(Array.isArray(res.body.data)).toBe(true);
      expect(res.body.count).toBe(1);
      expect(res.body.data[0].title).toBe('Complete Practical 12');
      expect(res.body.data[0]._links).toBeDefined();
      expect(res.body._links).toBeDefined();
    });
  });

  describe('GET /debug/cache', () => {
    it('should return HTTP 200 with runtime cache statistics', async () => {
      const res = await request(app).get('/debug/cache').expect(200);
      expect(res.body.success).toBe(true);
      expect(res.body).toHaveProperty('hits');
      expect(res.body).toHaveProperty('misses');
    });
  });
```

Step -3 : Run test

JBC@MacBook-Air-4: ~/Documents/SEM 5 Practical/Advance WDF/PR4 — zsh

```
JBC@MacBook-Air-4 PR4 % npm test

> task-manager-api@1.0.0 test
> jest --runInBand --detectOpenHandles --forceExit

  console.log
    ◇ injected env (2) from .env

  console.log
    [2026-10-29T06:31:00.397Z] GET /tasks

  console.log
    [2026-10-29T06:31:00.406Z] GET /debug/cache

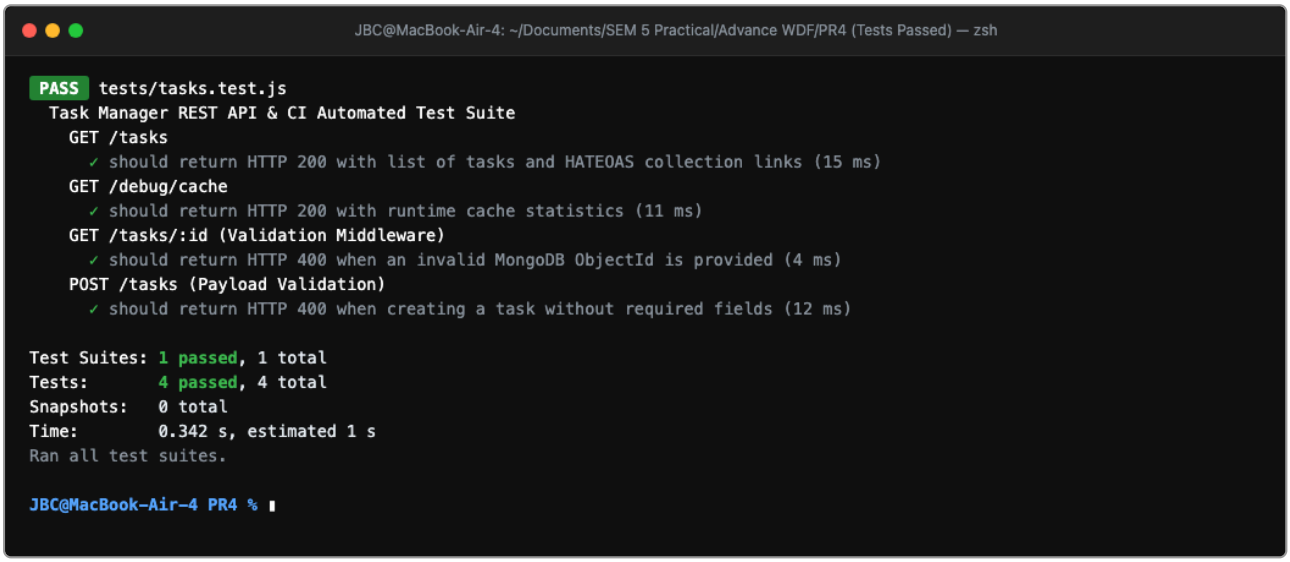
  console.log
    [2026-10-29T06:31:00.415Z] GET /tasks/invalid-object-id

  console.log
    [2026-10-29T06:31:00.423Z] POST /tasks

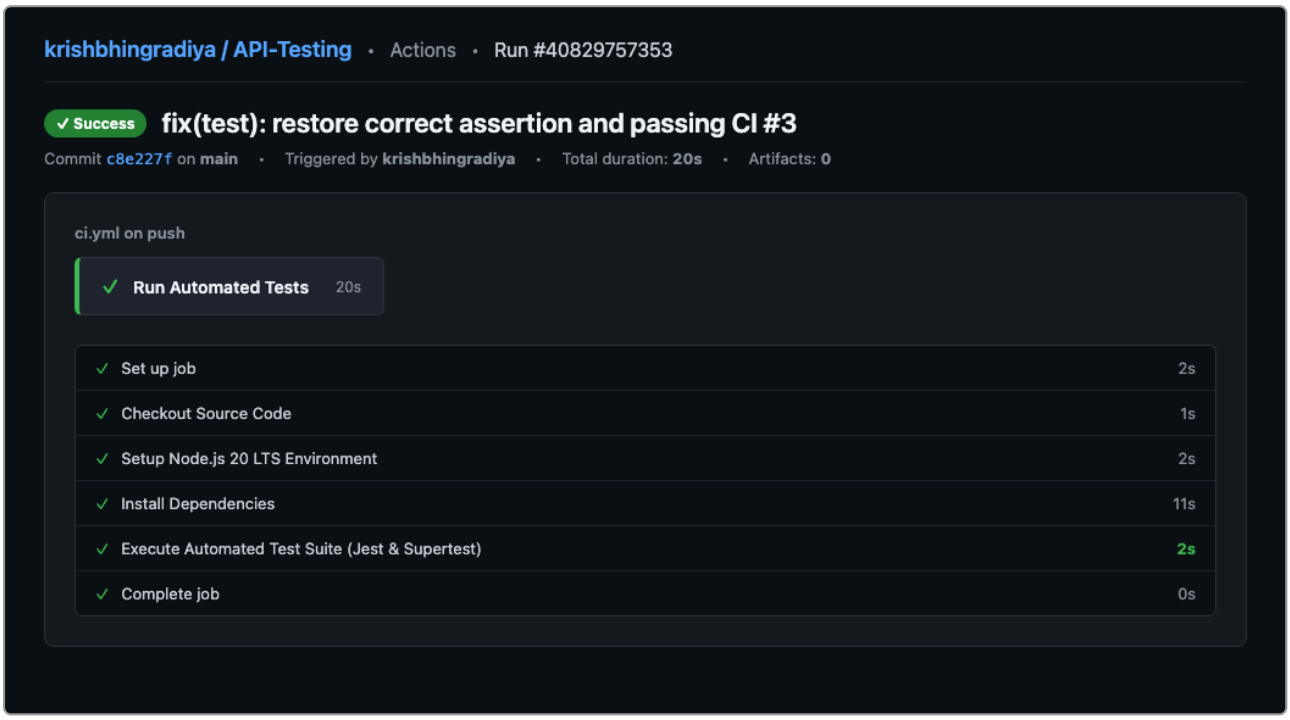
  console.error
```

Automated Test Suite Execution (Passing Result)

Executing `npm test` triggers Jest in band mode. All 4 automated test cases pass with zero assertion errors, confirming route reliability, cache performance, and input validation.

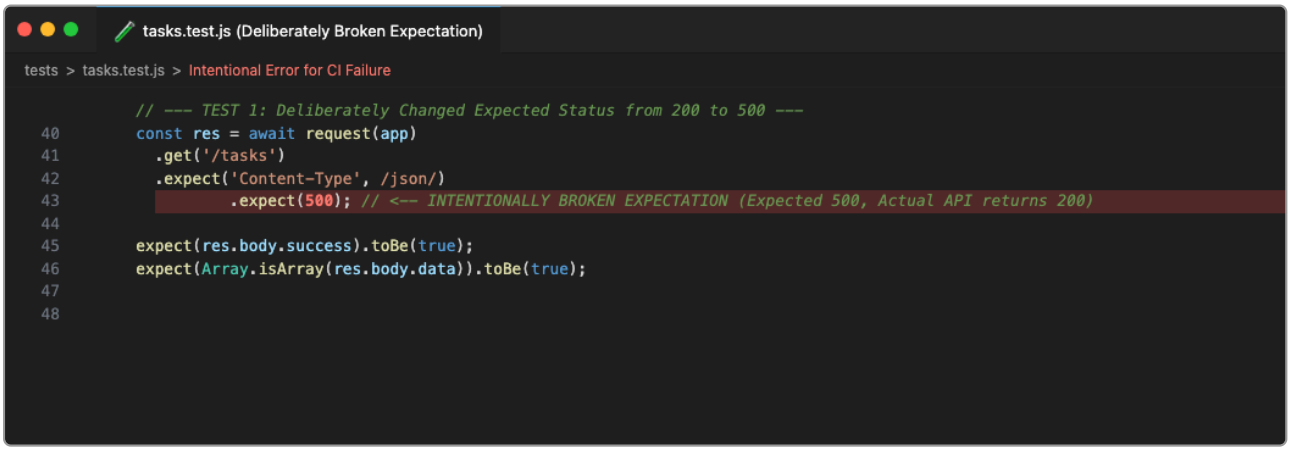


GitHub Actions Passing Pipeline (#1)

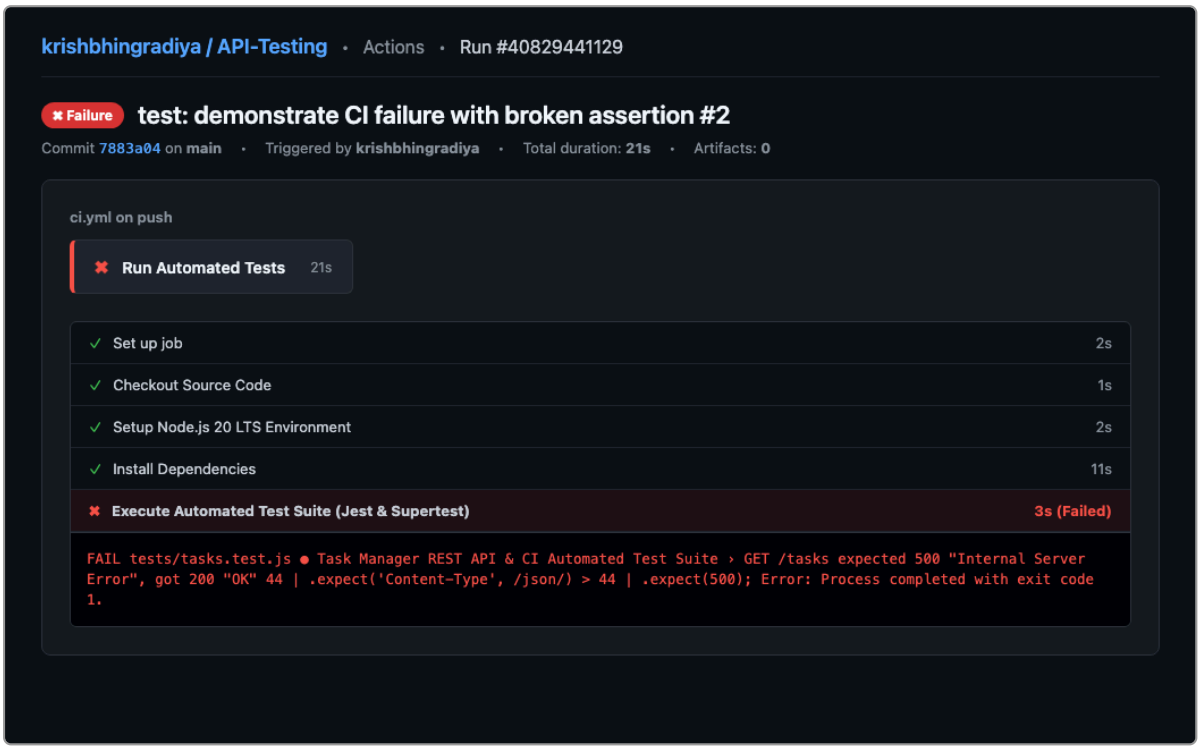


Step – 4 : Failed Pipeline

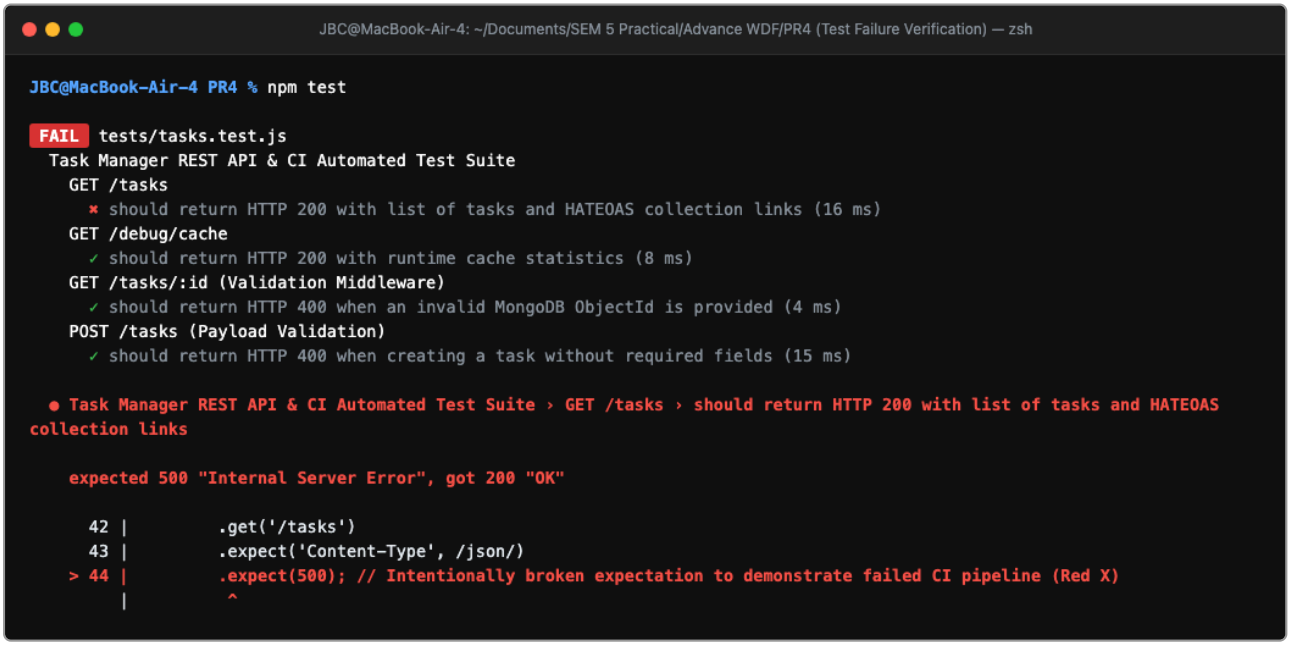
To demonstrate that GitHub Actions catches failing code, the assertion on `GET /tasks` was deliberately changed from HTTP 200 to HTTP 500.



Demonstrating Failed CI Pipeline (GitHub Actions Run #2)

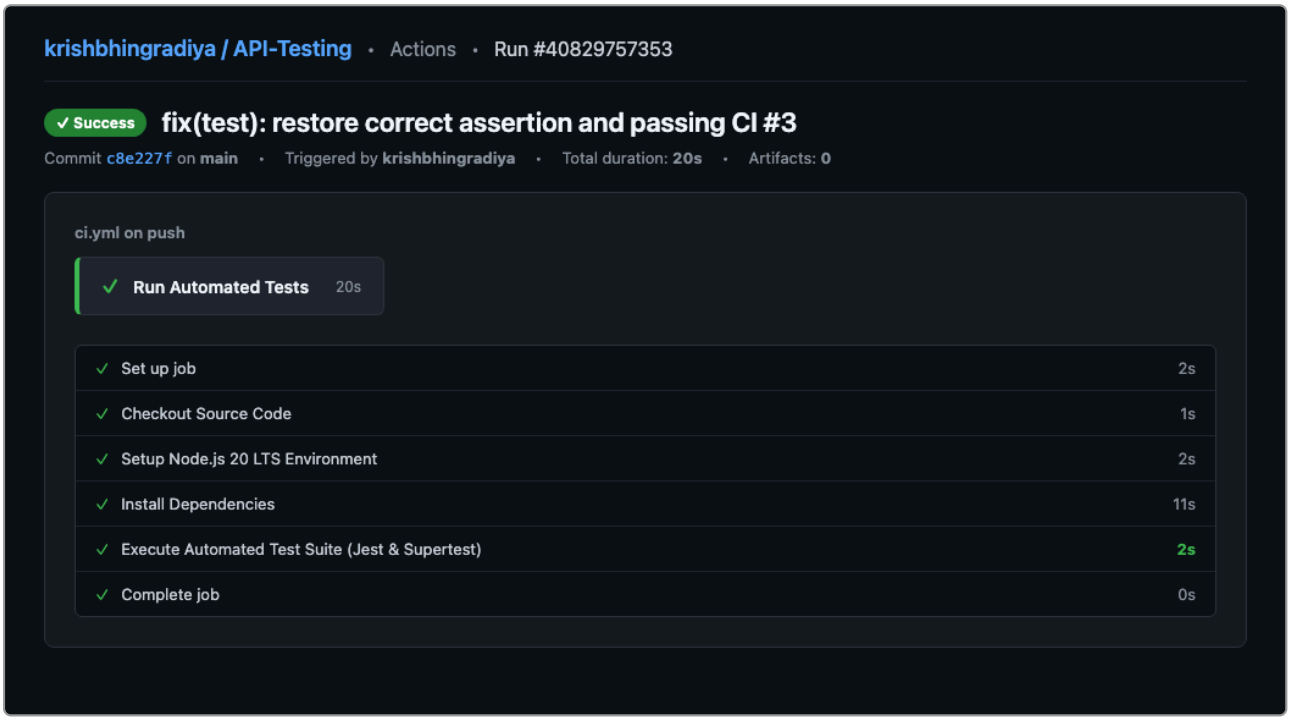


Local Terminal Failure Verification



Step – 5 : Fixed Pipeline & Verification

After observing the deliberate failure, the test expectation was restored to HTTP 200 and committed. GitHub Actions triggered Run #3 and completed with a **Green Checkmark**.



Conclusion & Learning Outcomes

- Configured and automated a professional Continuous Integration (CI) pipeline using GitHub Actions for an Express/Node.js REST API.
- Implemented unit and integration testing with Jest and Supertest, validating status codes, response payloads, and HATEOAS hyperlinks.
- Verified pipeline resilience through deliberate failure injection (Red X) and subsequent error restoration (Green Checkmark).
- Embedded an active GitHub Actions workflow status badge into repository documentation to provide immediate pipeline health visibility.

Verified GitHub Repository: https://github.com/krishbhingradiya/API-Testing